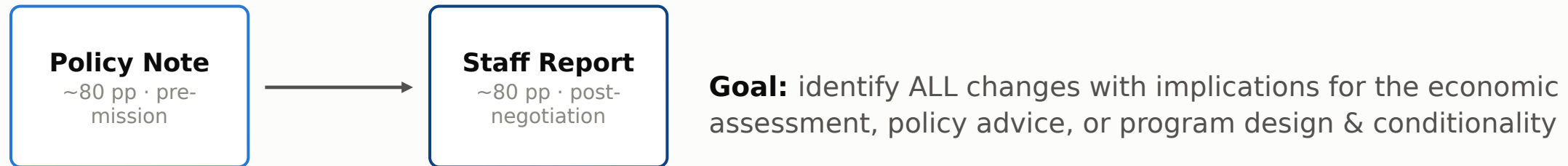


One Prompt vs. a Pipeline

Using AI to find every material change between an IMF Policy Note and its Staff Report — fiscal and debt policy focus



Compare two dense IMF documents — and miss nothing

Policy Note

Staff's intended strategy, written pre-mission

~80 pages

Complex fiscal, program and DSA tables

Staff Report

The post-negotiation outcome, presented to the Board

Different structure: Staff Appraisal, MEFP/LOI, program tables

Not simply “version 2” of the Policy Note

What “ALL changes” means here

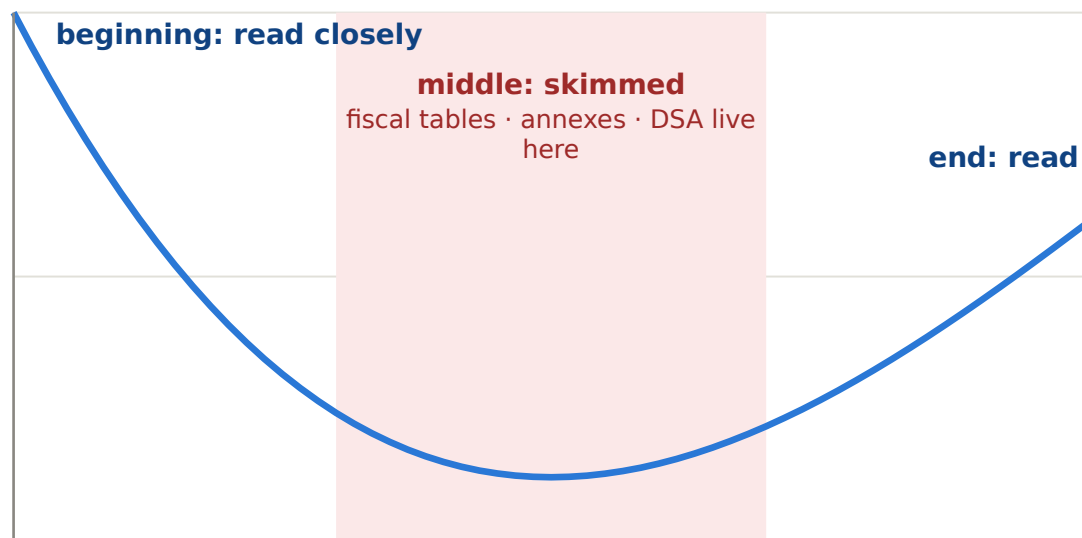
- Deviations, additions and omissions — in text, tables, footnotes and annexes
- Shifts in tone, strength, intensity and emphasis of the discussion
- Anything that could affect the economic assessment, policy advice, or program design and conditionality (fiscal & debt)

Read as an expert IMF economist would — not as a text diff.

“ALL” makes this a **recall problem**, not a summarization problem — and exhaustive recall is exactly where LLMs are weakest.

Long documents play against how LLMs read

Model attention



Position in an 80-page document

Changes hide in the worst places

Tables, footnotes and annexes — the content LLMs handle least reliably

Tone shifts are subtle

“Should” → “could” matters for policy; no mechanical diff detects it

Fluent ≠ exhaustive

LLMs are built to answer convincingly; completeness must be engineered

Microsoft's own guidance: on long files Copilot may focus on the beginning and ignore later content — split long documents and review them in parts.

Copilot doesn't “read” your PDF — it searches it



Advertised

Up to ~300 pages / 1.5 million words per file

Observed in practice

Text can silently drop from ~page 20-45, depending on density and app

Limits are undocumented and drift over time

No end-to-end reading → a footnote that contradicts a claim 60 pages later is structurally invisible to the model.

Five failure modes that matter for this task

Failure mode	What happens	Control
Hallucination MODEL	Claims a change or omission that isn't in either document	"No evidence, no finding" — every finding must quote both texts
Silent truncation PLATFORM	Parts of the file never reach the model at all	Controlled sections + coverage checklist, confirmed manually
Missed material changes BOTH	Subtle edits in middle sections, tables, annexes	Batching + global reconciliation + final sweep
Run-to-run inconsistency BOTH	Same prompt → different findings each run	Repeat runs; track stability; log prompts, versions, outputs
Tables & charts PLATFORM	Complex fiscal / DSA layouts parsed poorly	Extract tables separately; scripted, deterministic comparison

Source of the risk: **MODEL** = inherent to LLMs **PLATFORM** = Copilot product layer **BOTH**

Treat it like model validation, not magic

Banks already run this playbook for credit-scoring models: repeat testing for precision and recall, then human sign-off.

False positive · **Type I**

AI flags a change that isn't actually material

Cost: reviewer time to dismiss it

Annoying — but cheap and visible

False negative · **Type II**

AI misses a material change entirely

Cost: flawed assessment, wrong policy advice, reputational damage

Dangerous — silent and expensive

Design choice: tune the AI first pass for high recall (accept extra false positives) → human expert review restores precision.

One mega-prompt in Copilot Chat

THE PROMPT (condensed)

“As an expert IMF economist, compare the Policy Note and the Staff Report...

...identify ALL changes — deviations, additions, omissions, tone, strength, emphasis...

...extract everything, then prove 100% coverage.”

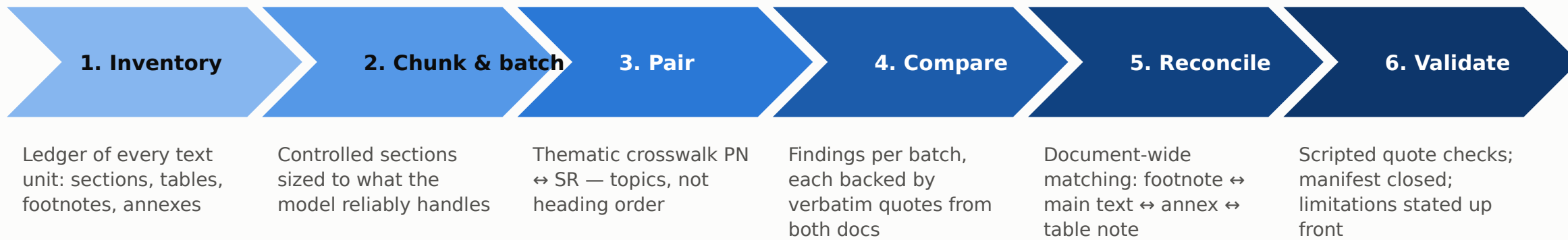
One message, both PDFs attached. Minutes to run.

Why “prove 100% coverage” cannot be honored

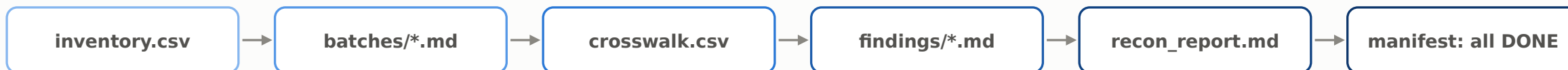
- 1 Output budget**
A reply has a hard length cap. The model silently compresses: 3 table rows “for illustration”, “similar revisions elsewhere...”
- 2 Retrieval is query-driven**
A diffuse mega-prompt makes a weak search query — entire tables never enter the model’s view at all
- 3 Instruction decay**
“Do not skip anything” fades as the context grows; hedges and shortcuts creep back in

Result: a fluent, confident memo built on the most salient ~30% of the documents — a smart skimmer with no audit trail.

A pipeline that forces coverage



Between every stage: a file on disk



Files carry the state, not the conversation — each stage starts a fresh, small chat context; a crashed or drifting session costs one step, not the whole run.

Why the pipeline looks like this

1. Files are the memory

Every intermediate product is written to disk. New chat per stage — the files carry the state, not the conversation window.

2. Thematic crosswalk, not structural alignment

A Policy Note and a Staff Report are not two versions of one document. Align by topic; coverage asymmetries are findings, not noise.

3. Determinism wherever possible

Section splitting, number extraction, pairing math, quote verification run as scripts with checkable output — model judgment only where it's needed.

4. Completeness by manifest, not by whim

A CSV lists every topic pair; every row must reach DONE; a final sweep pass catches anything the crosswalk missed.

Same engine underneath — different guarantees

	Copilot Chat (mega-prompt)	Structured pipeline
Process	Retrieve → analyze → synthesize, hidden and opportunistic	The same steps — explicit, forced, logged
Coverage	Unknown; whatever retrieval happened to surface	Manifest: every section pair visited
Evidence	Fluent prose; quotes unverifiable	Verbatim quotes, verified by script
Reproducibility	Varies run to run	Logged prompts, inputs and outputs
Effort	Minutes	Hours — and reusable for the next review

The pipeline turns an opaque, convenience-oriented process into a controlled, auditable QC workflow.

What the pipeline still doesn't guarantee

Coverage $\neq$ depth

The manifest proves every pair was visited — not read carefully. An agent can skim, write thin findings, tick the row.

The box says “done”, not “done well”.

Quote checks $\neq$ recall

Scripts catch fabricated evidence. They cannot catch an omitted finding — the tone shift the model never noticed.

There is no script for “what’s missing”.

Compliance is probabilistic

Agents drift: skipped steps, buggy extraction scripts, optimistic “complete”. First runs hit snags.

Expect run → fix → resume (cheap: stages are files).

The final control is unchanged: an expert economist. AI widens recall; the human supplies judgment and precision.

Practical lessons from the build

Digital PDFs need no OCR

Born-digital files carry a native text layer — parsing is fast and ~100% accurate. When visual/OCR checks were blocked, the limitation was declared up front, not papered over.

Narrow beats broad

One focused command per turn = a sharp retrieval query, a small output that fits the budget, and freshly re-anchored instructions.

Know the billing unit

Copilot charges per chat message (premium request), not per token — so the efficient pattern is few, well-loaded turns.

Leave a trail

Save prompts, dates, model/tool versions, inputs, outputs. Findings that aren't stable across runs get extra human review.

If you remember four things

1

Chat AI is a smart skimmer

Great for orientation and drafting — unsafe as the basis for “we found everything”.

2

Harnesses make thoroughness possible, not automatic

Left to its own judgment an agent satisfices; structure has to force coverage.

3

Tune the first pass for recall

A missed material change costs far more than a false alarm; human review restores precision.

4

Auditability is the product

Ledger, manifest and logs turn an opaque answer into a review someone can check — and defend.